

NIGHTWATCH

Autonomous Incident Commander Swarm

"Your AI incident commander for the 2 AM production failure."

Multi-agent production-incident response system | Project Design & Architecture Record

Field	Detail
Document type	Project design & architecture record
Prepared by	Vara
Status	Draft — pre-build architecture spec
Category	Hackathon / agentic systems project
Scope	Multi-agent incident detection, investigation, and remediation

This document is a structured record of the NIGHTWATCH project concept: an autonomous, multi-agent system that detects production incidents, independently investigates them across logs, code, and metrics, correlates the evidence into a root-cause hypothesis, and — after a mandatory human approval step — executes a remediation action such as a rollback. It is written to preserve the design rationale, architecture, and build plan for future reference.

Table of Contents

- 1. Executive Summary
- 2. Problem Framing & Positioning
- 3. System Architecture
- 4. Agent Roster & Responsibilities
- 5. Evidence Correlation & Root-Cause Scoring
- 6. Human-in-the-Loop Approval Flow
- 7. Technology Stack & Data Model
- 8. External Integrations
- 9. Demo Environment & Script
- 10. Validation Criteria ("How We Know It Works")
- 11. Build Plan — MVP vs. Full Version
- 12. Naming & Positioning Notes

1. Executive Summary

NIGHTWATCH reframes a common hackathon pattern — "an LLM reads logs and guesses the bug" — into a defensible multi-agent architecture: an **Incident Commander Swarm** that gathers evidence from independent systems, lets specialized agents investigate in parallel, correlates their findings into a scored hypothesis, and only proposes a production action after the evidence agrees. A human must approve any real-world action before it executes.

Within roughly 60–90 seconds of a triggered incident, the system aims to produce a root-cause hypothesis with a confidence score, a bundle of supporting evidence, and a single recommended action — surfaced in Slack with Approve / Reject / Investigate buttons.

Sample output the system should produce

```
Root cause: Payment-service deployment #8421 introduced a null-pointer
            error in PaymentValidator.java
Confidence: 91%

Evidence:
- Error rate increased 4 min after deployment
- 87 matching production errors found in Datadog
- PR #1842 modified the failing code path
- Same exception appears only on the new commit

Recommended action: Roll back deployment #8421
[ Approve Rollback ] [ Reject ] [ Investigate Further ]
```

The core design principle is that the agent never executes the rollback on its own initiative — a human approves, and only then does the system act.

2. Problem Framing & Positioning

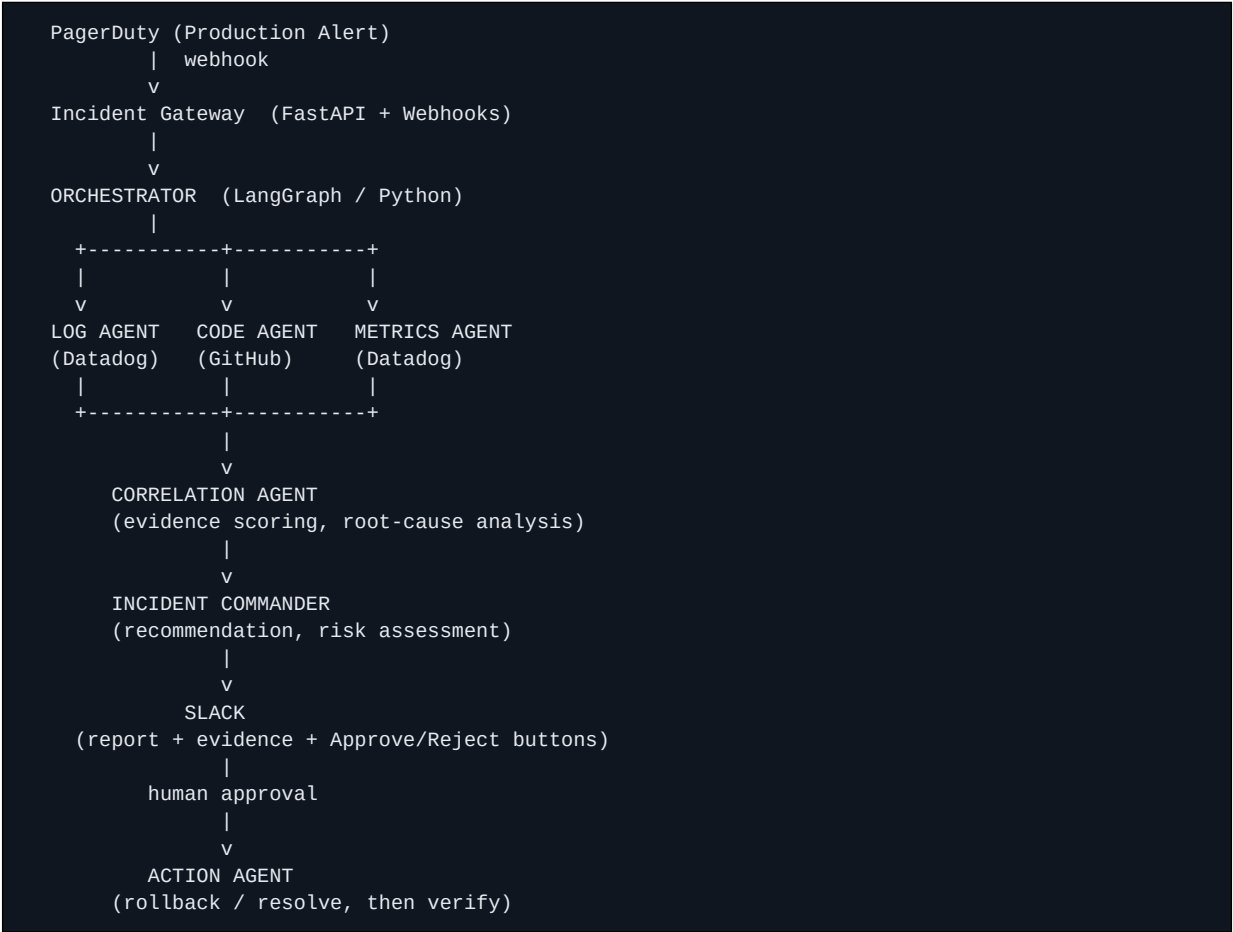
The central positioning decision for this project is to avoid presenting it as a single LLM call wrapped around an API ("alert → LLM → answer"), since that pattern is easy for judges or reviewers to see through. Instead, the system is framed and built as a genuinely multi-agent investigation, where each agent independently gathers evidence from a different system before a separate correlation step reconciles their findings.

Why this framing matters

- It forces a real multi-step, multi-tool architecture rather than a single prompt.
- It produces auditable, per-source evidence rather than an opaque LLM opinion.
- It naturally supports a human-approval gate before any production action.
- It maps cleanly onto a demo narrative: detect → investigate → decide → approve → act → verify → close.

3. System Architecture

The system is organized as a directed workflow: an inbound incident triggers parallel investigation agents, whose outputs feed a correlation step, which feeds a commander step that issues a recommendation, which is gated by human approval before an action agent executes and verifies the fix.



This satisfies a typical hackathon requirement of integrating three or more external systems, using four here: PagerDuty, Datadog, GitHub, and Slack.

4. Agent Roster & Responsibilities

Agent	Codename	Source system	Responsibility
Incident Investigator	Sentinel	PagerDuty	Parses the inbound incident: affected service, start time, error type, time window, suspected deployment.
Log Detective	Log Hunter	Datadog	Queries logs for the affected service/time window; extracts exception patterns, frequency, first occurrence, stack traces, affected endpoints.
Code Archaeologist	Code Detective	GitHub	Searches recent commits and PRs for the affected service; matches changed files/lines against the failing stack trace.

Agent	Codename	Source system	Responsibility
Timeline Detective	Timeline	Correlated	Builds a unified timeline of deploys, metric shifts, and first errors to establish before/after causality.
Correlation Agent	—	All agents	Aggregates evidence from all agents; scores competing root-cause hypotheses.
Incident Commander	Commander	Correlation Agent	Selects the top hypothesis, assesses risk, and issues a single recommended action with confidence score.
Action Agent	Operator	Slack approval	Executes the approved action (e.g., rollback) and verifies service health afterward.

4.1 Worked example across three agents

The Log Detective queries Datadog's Logs Search API (filterable by query string and time range) for the affected service and incident window, then deduplicates and extracts the dominant error pattern rather than passing raw logs to the LLM in bulk. The Code Archaeologist uses GitHub's REST API to search recent PRs and changed files on the affected service — not the whole repository — and matches them against the failing stack trace. The Timeline Detective then aligns both against deploy and metric events:

```
Log Detective -> NullPointerException x87, first_seen 11:43:21,
                  stack_trace PaymentValidator.java:142
Code Detective -> PR #1842 "Refactor payment validation" (Alex),
                  merged 11:39, changed PaymentValidator.java

Timeline        -> 11:39:02 PR #1842 deployed
                  11:40:11 payment latency +12%
                  11:41:03 5xx rate rising
                  11:43:21 first NullPointerException
                  11:44:07 PagerDuty incident triggered
```

Together these produce a causal chain: deployment → latency shift → errors → matching code change — the evidence bundle the Correlation Agent scores next.

5. Evidence Correlation & Root-Cause Scoring

The Correlation Agent is the reasoning core of the swarm. It receives structured evidence from every investigation agent and evaluates a small set of competing hypotheses, each scored by how well the evidence supports it.

Hypothesis	Supporting evidence	Score
Bad deployment	Deployment 2 min before incident; changed the affected service; changed the failing file; error appeared only after deployment	91%
Database failure	Some elevated DB latency observed; but no DB errors, and symptoms began before the incident window	23%
Traffic spike	No meaningful correlation with request volume	17%

The highest-scoring hypothesis ("bad deployment," 91%) becomes the Incident Commander's recommendation. This scoring layer is what elevates the system beyond a thin wrapper around an LLM call or a single API — it forces explicit, inspectable reasoning over multiple evidence sources before any recommendation is made.

6. Human-in-the-Loop Approval Flow

No remediation action is ever taken automatically. This is treated as a non-negotiable design constraint, both for safety and because it is the clearest way to demonstrate responsible agent design to an audience.



On approval, the Action Agent runs a verification sequence before acting:

- 1 Verify the approval came from an authorized source.
- 2 Verify the incident is still active (not already resolved).
- 3 Verify the deployment ID matches what was analyzed.
- 4 Execute the rollback.
- 5 Verify service recovery (error rate returns to baseline).
- 6 Update the Slack thread with the outcome.
- 7 Resolve the PagerDuty incident.

This sequence — detect, investigate, decide, obtain human approval, act, verify, close — is the complete agentic loop the project is built to demonstrate.

7. Technology Stack

Layer	Choice	Notes
Frontend (optional)	Next.js, TypeScript, Tailwind, WebSocket/SSE	Dashboard is a nice-to-have; Slack remains the real approval surface.
Backend API	FastAPI, Pydantic, httpx, asyncio	Handles inbound webhooks and orchestration entry points.
Orchestration	LangGraph	The workflow is naturally a graph — parallel investigation, aggregation, human-approval gate, action, verification — which fits LangGraph better than a linear chain.
LLM	Provider-agnostic (GPT / Claude / Gemini)	Wrapped behind a small interface so the underlying model is swappable without touching agent logic.

Layer	Choice	Notes
Primary datastore	PostgreSQL (SQLite acceptable for MVP)	Stores incidents, evidence, hypotheses, recommendations, approvals, actions.
Queue / state cache	Redis (optional)	Useful for job state, event dedup, and Slack approval state; skip if time is tight.
Deployment	Docker; Render / Railway / Fly.io; ngrok for local webhooks	Keeps the demo environment reproducible.

Design note on LLM choice

The specific LLM provider is treated as an implementation detail behind a thin interface (e.g., an `LLMProvider.analyze(...)` abstraction), so the model can be swapped without changing agent architecture. The differentiator for this project is not the model — it is the combination of tools, shared state, multiple independent agents, evidence aggregation, and a human approval gate.

7.1 Data model

A relational schema (PostgreSQL, or SQLite for the MVP) captures the full lifecycle of an incident so that every recommendation is traceable back to the evidence that produced it: an `incident` record (service, severity, status), linked evidence rows (source, type, content, confidence), scored `hypothesis` rows (description, score, supporting evidence), and action rows (type, approver, `executed_at`, result).

8. External Integrations

System	Purpose	Key implementation notes
PagerDuty	Incident trigger (entry point)	Use PagerDuty's V3 webhook subscriptions, which support <code>incident.triggered</code> plus acknowledgement, resolution, and escalation events. Route to a FastAPI endpoint (e.g. <code>/webhooks/pagerduty</code>), tunneled via ngrok in development.
Datadog	Logs and metrics	Requires <code>DD_API_KEY</code> and <code>DD_APP_KEY</code> with log-read permission. Query via the Logs Search API with service + time-window filters; deduplicate and summarize before passing to the LLM.
GitHub	PRs, commits, changed files	A fine-grained personal access token is sufficient for an MVP; a GitHub App is the production path. Use the pull-request APIs to list recent PRs, commits, and changed files rather than scanning the full repository.
Slack	Communication and human approval	Requires a Slack App with <code>chat:write</code> , <code>channels:read</code> , and <code>users:read</code> scopes. Use <code>chat.postMessage</code> with Block Kit, which supports interactive buttons, to post the incident report and receive Approve / Reject / Investigate actions at a webhook endpoint (e.g. <code>/webhooks/slack</code>).

9. Demo Environment & Script

Rather than depending on a real production incident occurring during a demo, the plan is to build a small, intentionally breakable microservice and a deterministic script to trigger a known failure — producing a repeatable, reliable demonstration.

Demo service structure

```
incident-demo/  
  payment-service/  
    app.py  
    payment_validator.py  
    Dockerfile  
  deploy/  
  load-generator/
```

The fault is injected deliberately — for example, calling a customer lookup and using the result before confirming it exists — so that a null value reliably produces a 500 error that Datadog captures and PagerDuty escalates.

Scripted demo flow

Scene	What happens
1. Normal world	Show the repository with a working deployment (e.g., v1.2) — everything appears healthy.
2. Inject failure	Run a trigger script that ships the bad code path. Errors begin appearing in Datadog; PagerDuty fires a P1 alert.
3. Swarm activates	UI/logs show agents spinning up in parallel: Incident Investigator, Log Detective, Code Archaeologist, Timeline Detective.
4. Evidence correlates	Correlation Agent combines findings: matching error count, PR that touched the failing file, deployment timing, absence of DB anomalies. Confidence settles around 91%.
5. Slack report	A structured incident report posts to Slack with root cause, confidence, evidence bullets, and Approve / Reject / Investigate buttons.
6. Human approves	Clicking Approve triggers authorization checks, incident-still-active checks, and a deployment ID match before rollback executes.
7. Verify and close	Health checks confirm error rate returns to 0%; Slack is updated with resolution and mean-time-to-recovery; PagerDuty incident is resolved.

10. Validation Criteria ("How We Know It Works")

To avoid resting the project's credibility on a subjective claim like "our AI is smart," the plan is to define a deterministic test scenario with explicit pass/fail checks, so the system demonstrably completes a known incident-response workflow.

Scenario: Bad payment deployment

Checks:

- | | |
|-------------------------------|-----|
| 1. Incident detected | [] |
| 2. Correct service identified | [] |
| 3. Correct error identified | [] |
| 4. PR #1842 identified | [] |
| 5. Root cause = deployment | [] |
| 6. Confidence > 80% | [] |
| 7. Rollback recommended | [] |
| 8. Human approval required | [] |
| 9. Rollback executed | [] |
| 10. Health restored | [] |

Passing all checks supports the stronger, more specific claim: the system correctly completed a known incident-response workflow end to end.

11. Build Plan — MVP vs. Full Version

Given limited build time, the recommended approach is to get the smallest possible end-to-end slice working first, then layer in the more impressive pieces.

11.1 Minimum viable slice

```
graph TD; A[PagerDuty webhook] --> B[FastAPI]; B --> C[LangGraph]; C --> D[Datadog logs]; D --> E[GitHub PR match]; E --> F[Root cause + confidence]; F --> G[Slack]; G --> H[Human approval]; H --> I[Mock rollback];
```

This slice alone — PagerDuty webhook → LangGraph → Datadog → GitHub → Slack approval — is enough to demonstrate the full concept and should be built and working before anything else.

11.2 Enhancements once the MVP works

- Timeline Detective agent for deploy-to-incident causality.
- Explicit confidence scoring across multiple hypotheses.
- Metrics correlation (latency, throughput) alongside logs.
- Real deployment rollback against a live environment (Kubernetes/AWS) rather than a mock.
- Automated health verification after remediation.
- Full incident resolution and PagerDuty close-out.
- A polished live dashboard showing agents working in parallel.

12. Naming & Positioning Notes

Two candidate names were considered for the system: **SWARM** ("Site-Wide Autonomous Response Mesh") and **NIGHTWATCH**. NIGHTWATCH was selected, framed around the tagline "Your AI incident commander for the 2 AM production failure." Giving the agents in-universe codenames reinforces the multi-agent framing and gives the project a memorable identity, rather than reading as a generic "AI chatbot for DevOps."

Role	Codename
PagerDuty interface	Sentinel
Log investigation (Datadog)	Log Hunter
Code investigation (GitHub)	Code Detective
Deploy/metric correlation	Timeline
Root-cause recommendation	Commander
Remediation execution	Operator

End of document. This record reflects the design as drafted prior to implementation and is intended as a reference for future build sessions rather than a finished specification.